Distributing Uneven Workloads Across Unequal Multi-Core Machines: A Measurement Study with a Distributed Connect Four Solver

Ahsan Baig and Sundas Habib
Department of Computer Science
National University of Computer and Emerging Sciences
Lahore, Pakistan
Corresponding author: sundashabib175@gmail.com

***Abstract*—Modern processors add cores instead of clock speed, and many now mix fast and slow cores on one chip. Ordinary computers also differ in age, cooling and power limits, so the speed of a machine is neither equal to that of another machine nor constant during a job. We study how to share a very uneven job between two such machines. The job is solving Connect Four on the standard board, where some tasks take milliseconds and others minutes. We built a distributed solver in which a coordinator hands tasks to a multi-threaded worker on each machine, and compared four ways of sharing: a fixed split by measured speed (V0), asking for work when free (V1), V1 plus splitting long tasks and cancelling unneeded ones (V2), and V2 plus sharing solved positions (V3). The machines were a 20-thread Intel Core i7-12800H laptop and a 16-thread AMD Ryzen 7 7700 desktop, linked by Wi-Fi or a direct cable. In 36 measured runs, all of which gave the correct result, V3 needed 66–74 s on average. This is about 1.8 times faster than the fixed split (118–137 s) and about twice as fast (2.0 to 2.3 times) as either machine alone using the same splitting (149 s and 153 s). The laptop's power limit moved between 76 W and 45 W during the session, and every unusually slow run coincided with the low limit. The network made no clear difference. Splitting the search made the machines examine many times more positions than one unsplit search, which solved the problem in 75 s on a single thread, so our best two-machine time only matched it. We report both comparisons.**

Keywords*—*multi-core processors; heterogeneous machines; load balancing; game-tree search; Connect Four; work splitting; distributed computing

I. Introduction

For many years the speed of a single processor core has grown slowly. Most of the extra performance of new computers comes from having more cores, and recent chips go one step further by mixing different kinds of cores. Intel's 12th-generation processors, for example, put fast "performance" cores and small power-saving "efficient" cores on the same chip. A program that runs on such a chip can no longer assume that every core works at the same speed.

The same unevenness appears between machines. A few ordinary computers can be joined into a small and cheap cluster, which suits students, small research groups and home users. Such a cluster is rarely uniform: machines bought in different years differ in core design, memory and cooling. A single machine does not even keep one speed. It may run fast while it is cool and slow down when it gets hot or reaches its power limit, and its operating system moves work between fast and slow cores while the program runs.

Some jobs make sharing much harder than others. In game solving, the aim is to find out who wins a game when both players play perfectly. The search is very uneven. One move can be settled in a blink while its neighbour needs minutes of searching, and nobody knows which is which before the search is done. Sharing such a job fairly is hard even on identical machines. On unequal machines whose speed changes, new problems arrive. A slow machine can be left holding one huge task while a fast one sits idle. A plan made from the speeds at the start goes wrong as the machines warm up. And the more finely the job is cut to keep every machine busy, the more likely it is that two machines search the same position.

Our literature review [1, 2, 3, 4] showed that existing work handles only part of this picture. Game-solving systems cope with very uneven tasks but run on identical, powerful servers. Load-balancing studies cope with machines or cores of different speed, but with even, predictable tasks. We found five open gaps:

1) real differences between machines are avoided or simulated;
2) machine speed is treated as fixed during a run;
3) uneven work and unequal machines are not studied together;
4) everyday hardware and networks are left out;
5) repeated work in small, unequal setups is not measured.

This paper addresses the five gaps together. Our question is: *how can unpredictable, uneven work be shared efficiently between a small number of unequal consumer machines whose speed changes while they work, and how much work is repeated when it is shared?* To answer it, we built a distributed Connect Four solver and ran it on two real machines of different make: a laptop with a hybrid 12th-generation Intel processor and a desktop PC with an AMD Ryzen processor. We compared four ways of sharing the work, each adding one idea to the one before, over Wi-Fi and over a direct cable, while logging the clock speed, temperature and power of both machines every second.

Our contributions are:

- an experimental study of uneven work on unequal machines with no simulated differences, with 36 measured runs that all gave the correct result;
- a step-by-step comparison of four work-sharing strategies, which shows what each added idea is worth;
- measurements that link the changing power limit of a laptop to how long runs take;
- a measure of how much extra work splitting causes, reported against two different references: the same method on one machine and a single unsplit search.

Section II reviews related work. Section III describes the methodology, and Section IV the implementation. Section V gives the results and Section VI the conclusions and future directions.

II. Related Work

A. Solving games on many machines

Čížek et al. [1] scale proof-number search on a cluster with a two-level design: a master keeps the global search, and each worker machine shares its job among threads. Workers keep what they learn between jobs and pass useful results on. On the game Sprouts, whose search trees are very lopsided, they report a speedup of 208.6 on 1024 cores, using 25 identical server nodes. They also found that workers who throw their results away after each job end up solving the same positions again and again. Wu et al. [2] use a manager and many workers, together with a neural network that is trained while the solve runs. On 7×7 Killall-Go, run on three identical GPU servers, they check 21.69% of the positions and use 23.54% of the time of the same solver without the trainer. Both systems deal with very uneven tasks, but both assume that the machines are equal and fast.

B. Sharing work between unequal machines and cores

Fernandes et al. [3] let each machine track how long its tasks take and how many it has left, and pass this to neighbours, so that machines take or steal the right amount of work. On a seismic simulation they gain about 10% over two other methods at 128 nodes, but the method works less well when each machine has only a few tasks. The speed differences are made by switching off cores on identical nodes, and the speed stays fixed. Beserra et al. [4] compare an equal split, a split by measured speed, and a split by measured speed with adjustment during the run, on one Intel Core i5-1235U laptop with a Monte Carlo workload. The speed-measured versions are clearly faster. The workload, however, is perfectly even.

C. Gaps

Table I summarises how the studies relate to our setting. None of them has uneven tasks, unequal real machines, speed that changes during the run and everyday hardware at the same time. This combination is what we study.

TABLE I. What the closest studies cover, compared with this work.

Study	Uneven tasks	Unequal machines	Speed changes during run	Everyday hardware
[1]	Yes	No	Not studied	No
[2]	Yes	No	Not studied	No
[3]	No	Faked	No	No
[4]	No	Cores only	Not studied	One laptop
This work	Yes	Yes	Yes	Yes

TABLE II. Research questions and the evidence used for each.

RQ	Question	Evidence
RQ1	How well can uneven work be shared between two machines that really differ in processor, memory and cooling?	Each machine alone (E1); all strategies together (E2)
RQ2	How much does each machine's speed change during a run, and how does this affect a split fixed at the start compared with sharing during the run?	Sensor logs; V0 compared with V1–V3
RQ3	Which strategy finishes soonest when task sizes are unknown and machines are unequal?	E2, V0 to V3
RQ4	Does ordinary Wi-Fi, compared with a direct cable, change the result?	E2 on both links; network checks
RQ5	How much extra work does splitting cause, and how do split size and sharing of results change it?	Position counts; E1, E3

III. Research Methodology

Our study is built on measurement. We build one solver, run it on two real machines, and compare four ways of sharing the work under conditions that we keep as steady as we can. Each research gap becomes a research question (Table II).

A. Experimental platform

We use two machines that differ in almost everything that matters for the first gap (Table III). The laptop has a hybrid processor, so it is uneven even inside itself. The desktop has eight identical cores but only one memory channel. Neither machine was slowed down on purpose; every difference comes from the hardware.

Both machines run the same Linux distribution (Omarchy, based on Arch Linux) with the same kernel and the same "performance" power profile and frequency governor, so the operating system is not a difference between them. The programs were built from the same source with the same compiler (GCC 16.2.1, `-O3`, static linking, the x86-64-v3 instruction set level that both processors support), and every program writes the commit, compiler and flags at the top of its log. No package updates were made between the code freeze and the last experiment.

Network. For the fourth gap the machines were joined in two ways: through a home Wi-Fi network (a 5 GHz network by its name; the radio frequency itself was not recorded), and through a direct gigabit cable with fixed addresses and no router. The same Wi-Fi network was used for every Wi-Fi run. During Wi-Fi runs the cable link was taken down on the

TABLE III. THE TWO MACHINES.

	Laptop 1	PC
Model	Dell Precision 5570	Desktop, MSI PRO B650-S WIFI board
Processor	Intel Core i7-12800H (12th gen)	AMD Ryzen 7 7700 (Zen 4)
Cores	6 performance (2 threads each) + 8 efficient: 20 threads	8 identical: 16 threads
Memory	32 GB DDR5-4800, two channels	16 GB DDR5, one channel, 4800 MT/s (rated 6000, profile off)
Power limit	Sustained limit set by the platform; moved between 76 W and 45 W; boost 115 W	Not readable under Linux (maker default 88 W, not measured)
Cooling	Laptop cooler at a fixed fan speed	Tower cooler
Network	Wi-Fi 6E; USB-C gigabit Ethernet adapter	MediaTek Wi-Fi; built-in 2.5 GbE port
System	Omarchy Linux, kernel 7.2.15, GCC 16.2.1	Same

TABLE IV. LINK MEASUREMENTS FROM THE NETWORK CHECKS OF 7 NOV 2026 (BEFORE THE PILOT), LAPTOP TO PC.

	Cable	Wi-Fi
Ping, median	1.52 ms	4.96 ms
Message round trip, median / 95th pct.	1.54 / 1.63 ms	2.41 / 4.71 ms
Message transfer rate	886 Mbit/s	204 Mbit/s
iperf3, there / back	941 / 941 Mbit/s	264 / 243 Mbit/s

laptop, so nothing could travel over the cable. Before the pilot we measured both links (Table IV).

B. Workload: solving Connect Four

We need a workload whose task sizes are very uneven and cannot be known in advance, and whose right answer is known so that every run can be checked. Connect Four on the standard 7×6 board fits both needs. It is a first-player win, and results for other board sizes have been published [5]. Pons [6] published a detailed guide to building a solver, with six test sets of 1,000 positions and the correct score of every position. The game tree is lopsided: some positions are settled after a handful of moves, others need an enormous search.

Solver. Each machine runs a solver that follows Pons's method [6]: negamax search with alpha-beta pruning, a compact bit representation of the board, a table of solved positions (the transposition table), and a move order that tries the most promising moves first. The solver only decides whether the player to move wins, draws or loses (a "weak" solver), which is enough to find the result of the game and allows early stopping. It runs one thread per hardware thread (20 on the laptop and 16 on the PC), all taking tasks from one shared pile and sharing one 1 GB transposition table, which is kept for the whole run. Threads are not tied to particular cores, because we want realistic conditions.

Turning the game into tasks. The coordinator expands the game tree from a start position down to a fixed depth d (the split depth). Every position at that depth becomes one task, written as the columns played to reach it (for example "4453"). The solver returns win, draw or loss for a task, and the coordinator combines the answers back up the tree. Different move orders can reach the same position, so some tasks are copies of each other: from the empty board, the 343 move sequences of length three reach 238 different positions, and the 2,401 sequences of length four reach 1,120. Mirror images are treated as different tasks.

TABLE V. THE FOUR WORK-SHARING STRATEGIES.

	Split decided	Work moves between machines	Long tasks split	Unneeded cancelled; results shared
V0	Before run, by speed	No	No	No; No
V1	During run, on request	Yes	No	No; No
V2	During run, on request	Yes	Yes	Yes; No
V3	During run, on request	Yes	Yes	Yes; Yes

C. Work-sharing strategies

We compare four strategies, each adding one group of ideas to the one before (Table V).

V0, fixed split by measured speed. Just before a V0 run, both machines solve the same 200 test positions, and their speeds are measured. The coordinator then deals the tasks out once, in proportion to the two speeds, and no work moves afterwards. The time spent calibrating is reported separately and is not part of the solve time.

V1, ask for more. Nothing is divided in advance. All tasks sit in one pile and a machine asks for more whenever it runs low, so the faster machine takes more tasks without its speed ever being measured.

V2, V1 plus splitting and cancelling. A task that runs longer than a time limit is stopped and replaced by its child positions one move deeper, which return to the pile. When one move of a position is found to win, the position is decided, and every task below it, waiting or running, is cancelled.

V3, V2 plus sharing results. The coordinator remembers every answer, merges tasks that are the same position into one, and does not send out a position it already knows. It also gives tasks that share a parent position to the same machine, so that the machine's table can be reused. If a position is being solved, a copy of it is held back and gets the answer when it arrives. This follows the finding of Čížek et al. [1] that keeping and sharing results removes much repeated work.

D. Experiments

Table VI lists the experiments. Every measured run was repeated three times. Everything else was held fixed: the start position, the program build, the number of threads, the table size, the power settings, the starting temperature and the background programs.

Pilot choices. A pilot study fixed the open settings before any measured run (Table VII). We wanted a start position that one machine needs a few minutes to solve, so that the slowdown we study happens inside every run. We tried shorter starts first on the laptop alone: the positions reached by the moves 444444, 44444, 4444, 444, 44 and 4 took 0.4, 3.1, 4.6, 4.8, 14.3 and 27.0 s, and the empty board took 116.8 s. None of the shorter starts was long enough, so we chose the empty board, the standard start with a published result. Fixed costs weigh more in short runs, so the single-machine runs last only

TABLE VI. EXPERIMENT PLAN AND NUMBER OF MEASURED RUNS.

Set	What changes	Runs
E0 Pilot	Settings being tuned; not reported as results	—
E1 References	Machine (laptop or PC) alone, with the features of V3 that help one machine; plus one single-thread unsplit search	$2 \times 3 = 6$, plus 1
E2 Main	Strategy (V0–V3) $\times$ link (Wi-Fi, cable)	$4 \times 2 \times 3 = 24$
E3 Depth	V1 and V3 at the second split depth, cable	$2 \times 3 = 6$

TABLE VII. SETTINGS CHOSEN IN THE PILOT (7 NOV 2026) AND FROZEN WITH THE CODE (COMMIT 6320703).

Setting	Value
Start position	Empty board (first player wins)
Split depth, E1 and E2	3 (343 tasks)
Split depth, E3	4 (2,401 tasks)
Time limit for V2, V3	40 s
V0 calibration	200 positions of Pons's set Begin-Medium (0.73 s laptop, 0.86 s PC)
Cool-down	At least 60 s, and CPU package at most 51°C (laptop), 52°C (PC)
Transposition table	1 GB on both machines

about two to three minutes, shorter than we first aimed for, and the two-machine runs about one to two minutes. The time limit for V2 and V3 was set to the 95th percentile of the 265 task times observed in a pilot V1 run (41.6 s), rounded to 40 s.

E. Procedure

The measured session ran on 7 Nov 2026 from about 15:45 to 18:20 on the frozen code; the room was at 29°C at the start and 25°C at the end, and the laptop stood on a laptop cooler at one fixed fan speed throughout. Before each run both machines rested for at least 60 s and until their CPU package temperature was below the pilot limit, so that a run did not start on a warm machine. The E1 runs came first, then three rounds of E2, then E3, and last the unsplit search. In each E2 round the cable block ran first and the Wi-Fi block second; the strategy order was V0 to V3 in rounds one and three and V3 to V0 in round two, and the E3 runs were ordered in the same alternating way. The laptop was on its charger, and nobody touched either machine during a run.

Every run writes four kinds of logs: the coordinator's event log, each worker's task log, each solver's position counts (every second) and a sensor log on each machine (every second: package temperature, package power, the clock of every logical CPU, and the laptop's power limits). The coordinator also measures the round trip and the clock difference to the PC every 10 s, so the logs of the two machines can be lined up. Raw logs are never edited; all tables and figures come from them by scripts.

F. Correctness checks

Speed only matters if the answers are right, so we checked at four points. (1) The solver returns the correct result for all six of Pons's test sets. (2) The system reproduces the published results for the 4×4, 6×4 and 6×6 boards [5]. (3) A stopped or cancelled task never stores a partly computed value in the shared table, and the table is safe for many threads. (4) In every measured run the result of the start position is compared with its known value. A run counts as invalid if it timed out, if the laptop's worker failed, if an unexpected worker took part, or if a solver reported a different build or table size.

G. Metrics

Solve time T is the time from the first task being sent until the result of the start position is known. With T_1 and T_2 the solve times of the laptop and the PC alone, the speedups are $S_1 = T_1/T$ and $S_2 = T_2/T$, and the combined efficiency is $E = T_{ideal}/T$ with $T_{ideal} = T_1 T_2/(T_1 + T_2)$, the time of a perfect split of the same work. We also report the positions searched (including stopped and cancelled tasks), compared with the laptop alone and with the single unsplit search; the tasks answered from stored results; and the clock, temperature and power of each machine during the runs. The number of positions does not depend on clock speed, so it is compared only between runs with the same setup.

IV. IMPLEMENTATION

A. Architecture

The system has three kinds of part (Fig. 1). The coordinator runs on the laptop. It builds the task tree, decides which machine gets which task according to the strategy under test, combines the answers as they arrive, and records every event with a time stamp: task sent, started, finished, stopped, split or cancelled. One worker runs on each machine, including the laptop, so both are treated the same way. A worker starts one solver process at the beginning of a run and keeps it until the end, so that start-up costs are paid once.

B. Solver

The solver is a C++20 program. It reads one command per line from its worker and answers one line at a time, so it never touches the network. The commands are `solve`, `stop`, `cancel` and `quit`; the answers include `started`, `result` (win, draw or loss with the positions searched and the time), `stopped`, `cancelled` and a `progress` line every second. Every task gets exactly one final answer.

A shared transposition table is the main data structure. Each slot is a single 64-bit word holding part of the position key and the stored value, and it is read and written in one atomic step, so a thread can never mix the key of one position with the value of another. The number of slots is a prime, which together with the stored key part guarantees that two different positions cannot be confused. When a task is stopped or cancelled, every level of its search returns at once without storing anything, because the half-finished values would be wrong and would poison other tasks.

A position is counted every time the search function is entered. Each thread has its own counter, so threads do not slow each other down, and a separate thread adds the counters up once a second.

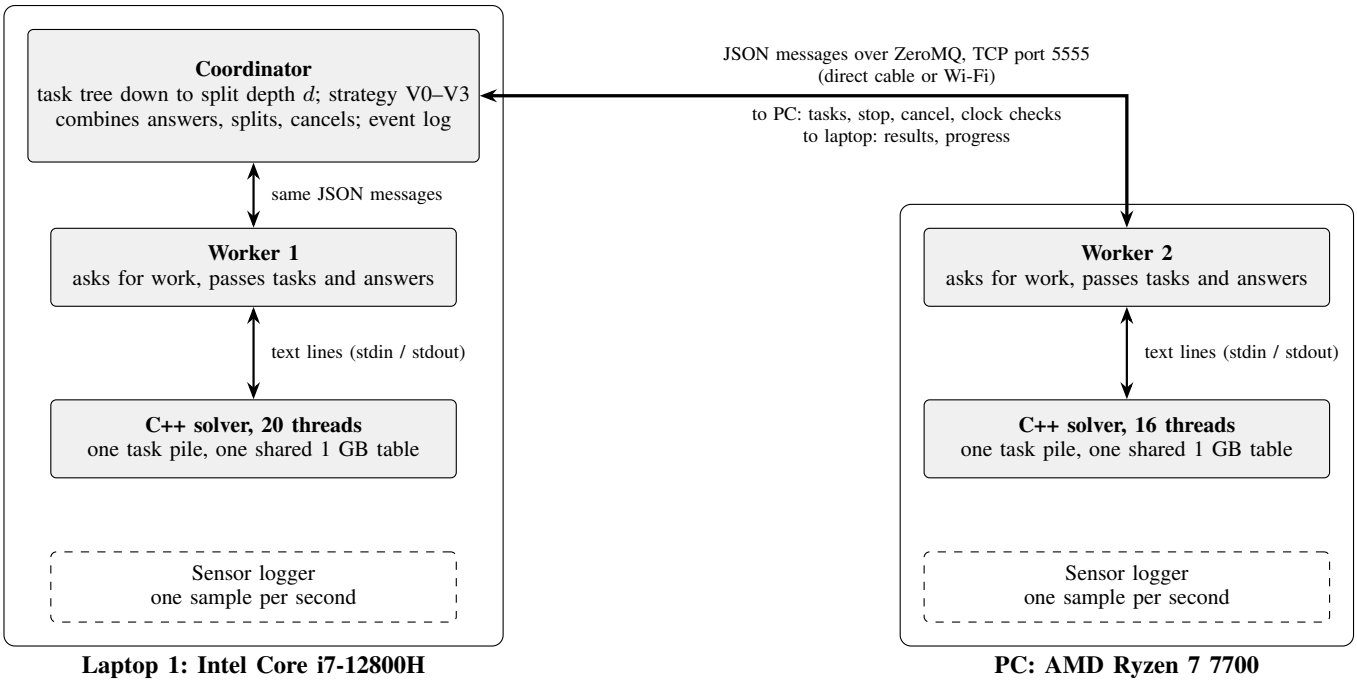


Fig. 1. System architecture. The coordinator on the laptop holds the task tree and applies the strategy; a worker on each machine drives a multi-threaded solver with one shared table; the machines talk over a direct cable or Wi-Fi. Both workers speak the same messages to the coordinator, and each machine logs its own sensors.

C. Worker and coordinator

Coordinator and workers exchange JSON messages over ZeroMQ on TCP port 5555. The coordinator uses a ROUTER socket and each worker a DEALER socket, so the coordinator can send `stop` and `cancel` while a worker is busy. Every message carries its type and a protocol version. A worker introduces itself with the commit, the uncommitted-changes flag and the table size of its solver, so the coordinator can spot a worker built from other code.

Work is handed out by credit. A worker starts with a credit of two tasks per thread (one running and one waiting) and adds one credit each time a task ends. The coordinator never sends more tasks than the credit allows, so a thread never waits for a task to travel over the network, and a machine cannot hoard tasks that it has not started. Both machines use the same code for all four strategies; only the coordinator's choice of tasks differs.

D. Strategies in the coordinator

V0 sends both machines the calibration set at once, measures how long each takes to solve it, and deals the tasks out in turn, in proportion to those speeds. **V1** keeps one pile and sends a task to whichever worker has credit. **V2** adds a timer for each running task: after the limit the coordinator sends `stop`, expands the task one move deeper and puts the children back in the pile; when a winning move is found it sends `cancel` for all tasks below the decided position, and the decision may pass up the tree.

V3 needs no change to the protocol. The coordinator's tree passes every answer on to all copies of the same position. A task whose position is being solved is held back; a known position is never sent; and a running task is kept, not cancelled, while a held copy elsewhere in the tree still needs its answer. Tasks with the same parent go to the same machine in groups, children of a split task go to the machine that held it, and a machine with nothing left takes tasks from the back of the other machine's queue, so it never idles while work waits. In the one-machine runs (E1) the same code runs with one worker.

E. Sensor logging and run control

A small logger reads the Linux sensor files once a second: package temperature, package power from the processor's energy counter, the clock of each logical CPU, and, on the Intel laptop, the power limits and thermal-throttle counts. Its logs use the same two clocks as the run logs, so they line up with the runs. A run script checks that the build is the release build of the frozen commit with no uncommitted changes, that the laptop is on its charger and that the sensor log is current, waits for the cool-down, starts the run, and marks it invalid if it fails the rules above. Settings are kept in a committed file, so a run refuses to start if anything differs from the frozen values. The code has 229 automated Python tests and 11 C++ checks that run on both machines, and the small-board checks pass on both.

V. Results and Discussion

All 36 measured runs were valid and returned "win" for the empty board, as expected. No run was dropped or repeated. One run (V0, Wi-Fi, first repeat) waited about 35 minutes for the PC to join, because of a bug in a helper script on the PC, but its calibration and solve began only after both machines had joined and rested, so it is kept as a normal run. For every

TABLE VIII. REFERENCE RUNS: SOLVE TIME IN SECONDS.

Setting	Mean (lowest–highest) of 3 runs
E1, laptop alone (20 threads, split search)	149 (123–200)
E1, PC alone (16 threads, split search)	153 (153–155)
Unsplit search, one thread of the laptop	75.3 (1 run)

TABLE IX. E2: SOLVE TIME IN SECONDS, DEPTH 3: MEAN OF THREE RUNS (LOWEST–HIGHEST).

Strategy	Cable	Wi-Fi
V0 fixed split	137 (124–161)	118 (113–121)
V1 ask for more	124 (108–150)	131 (112–153)
V2 + split, cancel	84.7 (79–88)	76.7 (74–80)
V3 + share results	73.9 (67–86)	66.5 (66–68)

setting we give the mean of the three runs and the lowest and highest run, and a difference smaller than the run-to-run spread is not claimed as a finding.

A. Each machine alone

Table VIII gives the reference times. The laptop alone took 149 s on average, but with a wide spread: two repeats took about 123 s and the third 199.9 s. In the third repeat the laptop's power limit stayed at 45 W for the whole run (76 W in the first repeat), its package power averaged 48 W against 71 W, and the slower machine made the 40 s time limit fire more often: 16 tasks were split instead of 3, and the run searched about 25% more positions (23.9 billion against 19.0 and 19.1 billion). The PC alone took 153 s, with almost no spread. The single unsplit search on one laptop thread examined 1,049,291,061 positions in 75.3 s. It is the only unsplit run, so it has no spread.

B. Comparing the four strategies

Table IX and Fig. 2 show the main comparison. In the means the strategies come out in the same order on both links, with one exception: V1 was slower than V0 over Wi-Fi. The step from V0 to V1 is not clear, though. Their ranges overlap, and V1 was faster than V0 in only three of the six pairs of runs from the same round and link; its two slowest runs (the third repeat on each link) were among those at the low power limit described below. V0 and V1 hardly beat the laptop alone. V2 was clearly faster than V1: its slowest run (87.6 s) was faster than V1's fastest (107.7 s), and on average it cut V1's time by about a third to two fifths. V3 was faster than V2 in five of the six pairs, and on average cut V2's time by about a further 13%. The exception was the third cable round, in which V3 was one of the runs held at the low power limit.

C. Speedup against two references

Table X gives the speedups and the combined efficiency of every run, as the mean of the three runs with the lowest and highest value. Against one machine alone running the same split search (E1), V3 is about 2.0 times faster than the laptop over the cable and 2.2 times faster over Wi-Fi, and slightly more against the PC. Against the unsplit single-thread search the picture is different: V0 and V1 are slower than it, V2 is about as fast, and V3 is at best 1.13 times faster (last column). Both comparisons are valid, and readers should keep both in mind. The ideal time of a perfect split of the E1 work, $T_{ideal} = 75.5$ s, happens to be close to the unsplit time, so E and the last column tell almost the same story. E is above 1 for V3 because V3 searches fewer positions than the E1 runs did (Table XI). The laptop's E1 mean is raised by its slow third repeat. With only its two faster repeats (about 123 s), the V3 speedup over the cable would be about 1.7 instead of 2.0.

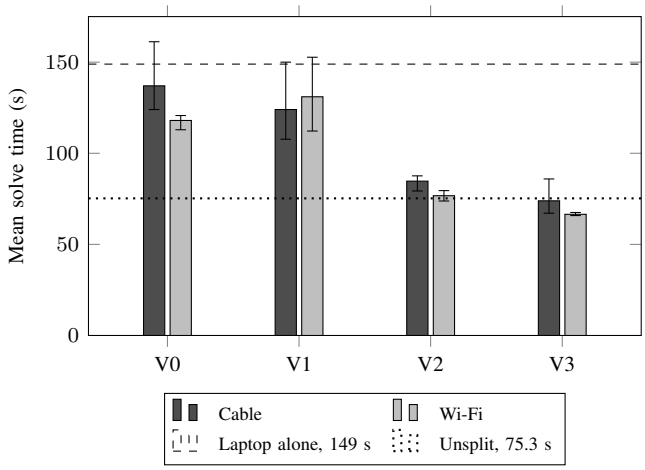


Fig. 2. Solve time of the four strategies over the cable and Wi-Fi, against two references. Each bar is the mean of three runs; the whiskers show the lowest and highest run.

TABLE X. SPEEDUP OVER THE LAPTOP ALONE (S_1) AND OVER THE PC ALONE (S_2), COMBINED EFFICIENCY E, AND SPEEDUP OVER THE UNSPLIT SEARCH (75.3 S DIVIDED BY THE MEAN TIME). MEAN OF THREE RUNS; THE LOWEST AND HIGHEST RUN ARE IN BRACKETS.

Strategy	Link	S_1	S_2	E	vs unsplit
V0	Cable	1.10 (0.92–1.20)	1.14	0.56 (0.47–0.61)	0.55
V0	Wi-Fi	1.26 (1.23–1.32)	1.30	0.64 (0.63–0.67)	0.64
V1	Cable	1.22 (0.99–1.38)	1.26	0.62 (0.50–0.70)	0.61
V1	Wi-Fi	1.15 (0.97–1.33)	1.19	0.59 (0.50–0.67)	0.57
V2	Cable	1.76 (1.70–1.87)	1.82	0.89 (0.86–0.95)	0.89
V2	Wi-Fi	1.94 (1.87–2.01)	2.00	0.99 (0.95–1.02)	0.98
V3	Cable	2.04 (1.73–2.22)	2.10	1.03 (0.88–1.12)	1.02
V3	Wi-Fi	2.24 (2.20–2.26)	2.31	1.14 (1.12–1.15)	1.13

D. Machine speed changes during a run

The laptop's speed was not steady. Its sustained power limit is set by the platform and moved between 76 W and 45 W, apparently with temperature; we left it as the platform set it, because speed changes during a run are one of the effects we study. The sensor logs record both values every second. Figure 3 compares two V3 runs over the cable. In the first (68.7 s) the laptop's package power eased down from about 90 W to about 70 W, and it searched about 170 million positions per second against about 130 million for the PC. In the third round (85.9 s) the laptop's power fell to 45 W after 18 s and stayed there. Its clock dropped from about 3.9 to about 2.9 GHz and its speed fell to about 120 million positions per second, below that of the PC, which did not change. The laptop reached 100°C in every run, while the PC peaked between 55 and 63°C and drew 64 to 75 W.

Six of the 24 E2 runs held the low limit for the whole run, as did the laptop's third E1 repeat. Four of those E2 runs were the slow outliers: the second V0 run and the third V3 run

over the cable, the third V1 run over the cable, and the third V1 run over Wi-Fi. The other two (the second V0 run over Wi-Fi, 120.0 s, and the first V1 run over the cable, 114.9 s) were typical for their strategy, so the low limit does not always make a run slow, and our data do not say why. In these six runs the laptop's package power averaged 49 to 51 W, in the other 18 runs 64 to 75 W. V2 never spent a whole run at the low limit. Every unusually slow run coincided with it, but we did not control the limit, so we report this as an association.

This is what the second research gap predicted. A fixed split decided from a speed measured at the start (V0) assumes a speed that does not hold. In the six V0 runs, the ratio of the laptop's speed to the PC's speed measured in the calibration (1.10 to 1.29, from a set that takes under a second) differed from the ratio reached during the run (1.00 to 1.31) by between -12% and $+24\%$. The largest error, the second V0 run over Wi-Fi, was a run at the low limit in which the calibration overrated the laptop. The adaptive strategies simply give less work to a machine that asks less often. The run-to-run spread caused by the laptop's power limit is also larger than some of the differences between strategies and links, so those differences cannot be called findings.

E. The desktop was not the faster machine

On paper the desktop (eight identical cores with 16 threads, and a desktop cooler) looks at least as strong as the laptop. Yet alone it was slower (153 s) than the laptop at its full power limit (about 123 s), and in a shared run it searched about 130 million positions per second against about 170 million for the laptop (Fig. 3a). We did not test why. One possible reason is its single memory channel, which gives it about half the memory bandwidth of the laptop, and the large transposition table depends on memory speed. We report this as a hypothesis only.

F. Cable against Wi-Fi

The network cost little. The median round trip of the coordinator's clock checks in a run was 0.8 to 1.6 ms on the cable and 0.8 to 9.3 ms on Wi-Fi (on average 1.9 to 4.2 ms for the four Wi-Fi settings), against runs of 60 to 260 s, and the messages are tiny. Wi-Fi was also less steady than the cable: ping to the PC averaged about 5 ms at the network check, about 21 ms before the first two Wi-Fi rounds and 52 ms (25 to 101 ms) before the third, yet the fastest V3 run of all was a Wi-Fi run of the third round (65.8 s). Wi-Fi was faster than the cable in three of the four strategies, which is the opposite of what the network would cause. Three of the four slow E2 runs listed above were cable runs, so the differences between the links are consistent with the laptop's power state and the time of each block, not with the network. Without the cable's slow third V3 run, V3 took 67 to 69 s on the cable and 66 to 68 s on Wi-Fi. Over the three repeats the data do not show a clear effect of the link, so we answer RQ4 with "no clear difference".

G. Work done, wasted and repeated

Table XI shows how many positions each setting searched. Every split run searched between 16 and 68 times as many positions as the unsplit search (1.05 billion). Splitting the tree into independent tasks loses most of the pruning that makes alpha-beta search fast, because a task does not know the bounds found by other tasks. Our measurements show how large this cost is on two unequal consumer machines. The limit is clear: the best two-machine time (66–74 s) is about the time of one unsplit thread (75 s).

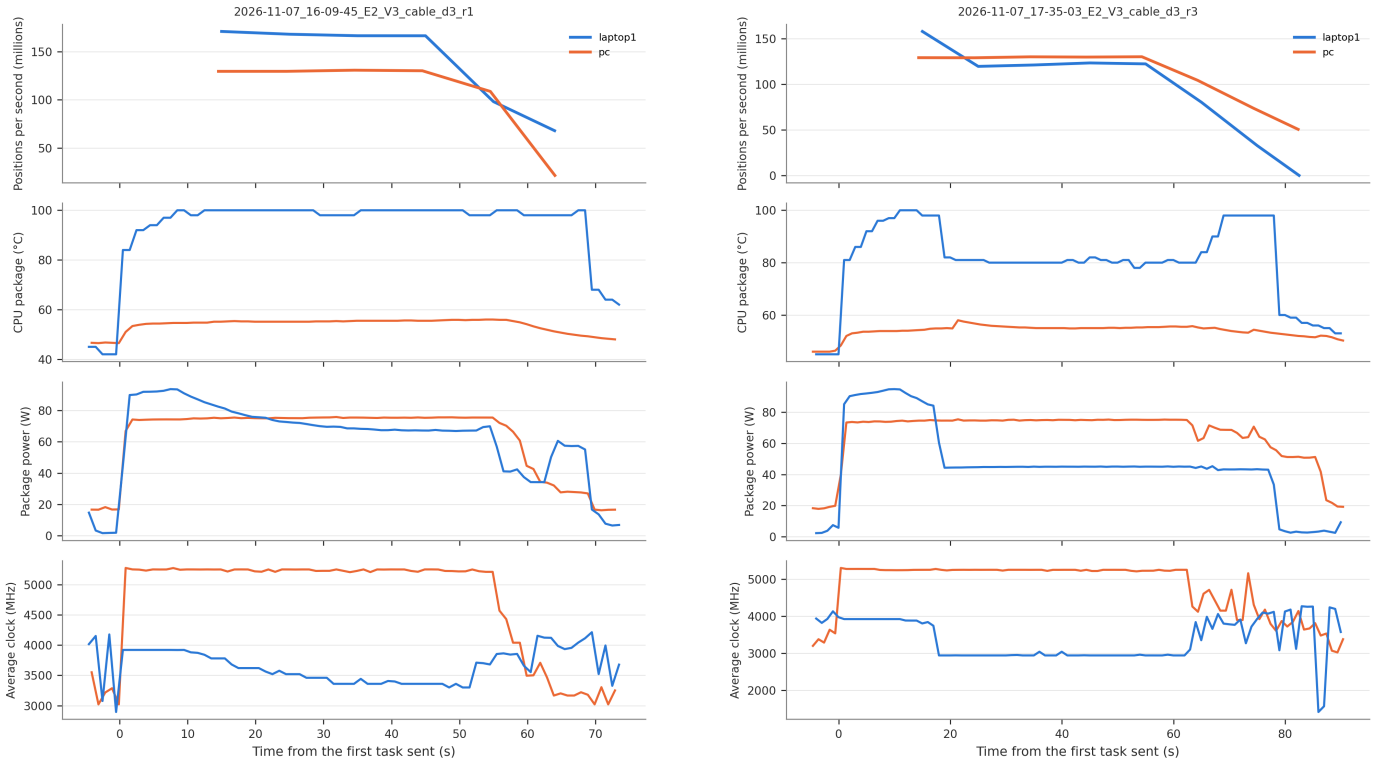
Three kinds of wasted work can be told apart. First, V0 and V1 never cancel a task. When the result of the start position was known, their machines had spent 43 to 50% of all their positions (74% for V1 at depth 4) in results that arrived when they could no longer change anything. V2 and V3 cancel such tasks and have none. Second, stopping and cancelling costs positions too: V2 lost 48% of its positions in stopped or cancelled tasks and V3 35 to 39%, against 19 to 23% for V0 and V1. Added together, the share of positions that went into stopped, cancelled or unneeded tasks was 66 to 70% for V0 and V1, 48% for V2 and 35 to 39% for V3. Third, copies of the same position: 105 of the 343 tasks at depth 3 (1,281 of 2,401 at depth 4) repeat another task. V0 and V1 sent 81 to 105 of these copies and searched them again, about 20 to 24% of their positions. V2 sent 64 to 72 (about 12% of its positions). V3 sent none or one, because it holds a copy back while its position is being solved and answers it from stored results: per run it answered 77 to 79 tasks from stored results and held back 46 to 54 copies. At depth 4, V1 sent 428 to 501 copies (about 32% of its positions) and V3 none.

Idle thread time shows why busy time is a poor measure here. V0 and V1 almost never left a thread idle (on average under 1% on the laptop and under 6% on the PC), yet they were the slowest strategies, because the threads were busy with unneeded and repeated work. V2 and V3 left 9 to 23% of the thread time idle, since after cancelling there is not always a task for every thread.

The time limit matters as well. In the pilot, V3 with the 40 s limit split only three tasks and searched 0.57 of the positions of V1. An earlier local trial with a 3 s limit, run while other programs were using the processor and therefore not a result, suggested that a limit that is too short makes V3 lose most of its work in stopped tasks; this is why we took the limit from observed task times. A slow machine also triggers the limit more often, as in the laptop's slow E1 repeat.

H. Split depth

Splitting one level deeper gives 2,401 tasks instead of 343 (Table XII). For V1 this doubled the time (261 s against 124 s over the cable). V3 at depth 4 (140 s) was 1.9 times slower than at depth 3, but still 1.9 times faster than V1 at depth 4; its lead over V1 grew slightly, from 1.7 times at depth 3. V3 at depth 4 took about as long as V1 at depth 3. The extra work explains this (Table XI): at depth 4 V1 searched 65 times the unsplit positions (36 times for V3), three quarters of them in results that arrived too late to matter. A finer split has more copies of the same position (1,120 distinct positions among 2,401 tasks) and probably loses more pruning, so it does not pay off here. V1 at depth 4 also ran with the laptop at the low limit for 62 to 74% of each run, so the depth effect is partly mixed with the laptop's state.



(a) V3, cable, first round: 68.7 s

(b) V3, cable, third round: 85.9 s, laptop held at 45 W

Fig. 3. Speed, CPU package temperature, package power and average clock of both machines in two runs of the same strategy and link. Time is counted from the first task sent. The blue line is the laptop and the orange line the PC.

TABLE XI. WORK DONE. POSITIONS SEARCHED (BILLIONS), AS A MULTIPLE OF THE UNSPLIT SEARCH (MEAN, WITH THE LOWEST AND HIGHEST RUN IN BRACKETS), SHARE OF POSITIONS IN STOPPED OR CANCELLED TASKS (LOST) AND IN RESULTS THAT ARRIVED WHEN THEY COULD NO LONGER MATTER (UNNEEDED), AND COPIES OF AN ALREADY SOLVED OR RUNNING POSITION THAT WERE SENT (MEAN).

Setting	Positions (billions)	× unsplit	Lost (%)	Unneeded (%)	Copies sent
E1 laptop alone	20.7	19.7 (18.1–22.8)	40	0	0
E1 PC alone	19.0	18.1 (18.0–18.1)	37	0	0
V0 cable	36.9	35.2 (34.2–36.8)	19	47	97
V0 Wi-Fi	33.3	31.7 (29.3–34.1)	20	50	96
V1 cable	32.5	30.9 (27.7–36.5)	22	47	92
V1 Wi-Fi	36.0	34.3 (31.5–37.3)	23	43	104
V2 cable	21.8	20.7 (19.3–21.6)	48	0	69
V2 Wi-Fi	20.4	19.4 (19.0–19.7)	48	0	66
V3 cable	18.1	17.3 (16.7–18.2)	39	0	0
V3 Wi-Fi	17.2	16.4 (16.2–16.6)	35	0	0.7
V1 cable, depth 4	68.5	65.3 (63.6–68.5)	4	75	456
V3 cable, depth 4	37.8	36.0 (35.4–36.5)	39	0	0

TABLE XII. E3: SOLVE TIME IN SECONDS OVER THE CABLE AT THE TWO SPLIT DEPTHS: MEAN OF THREE RUNS (LOWEST–HIGHEST).

Strategy	Depth 3 (343 tasks)	Depth 4 (2,401 tasks)
V1	124 (108–150)	261 (252–277)
V3	73.9 (67–86)	140 (133–149)

I. *Answers to the research questions*

RQ1 and RQ3. The two machines really differ, and a plan fixed at the start used them badly: the fixed split (V0) gained only 1.1 to 1.3 times over one machine alone. Splitting long tasks and cancelling unneeded ones (V2) and sharing results (V3) did much better, and V3 was the fastest strategy on both links, about 1.8 times faster than V0 and about twice as fast as either machine alone. Asking for work when free (V1) was not clearly better than the fixed split on its own. The combined efficiency of V3 (1.03 and 1.14) shows that, once wasted work is cut, two unequal machines share the work about as well as a perfect split of the single-machine work would.

RQ2. The laptop's speed moved with its power limit, and every slow run coincided with the low limit, so a speed measured at the start did not describe the run: the calibration of V0 was off by -12% to $+24\%$. Sharing during the run was more robust than the fixed split, but it did not remove the effect: the slow V3 run took 85.9 s against 67 to 69 s for the other two over the cable.

RQ4. The network was not the bottleneck at this task size: round trips of 1 to 4 ms did not matter against runs of a minute or more, and even a Wi-Fi link with ping times of 25 to 101 ms did not slow a run.

RQ5. Splitting caused very large extra work (16 to 68 times the unsplit search). Cancelling unneeded tasks, splitting long tasks and sharing results reduced it, with V3 doing the least work, and a finer split made it worse. Sharing results removed the repeated work completely (one copy sent at most, against 64 to 105 for the other strategies).

J. Limits of the study

The results describe one start position, three repeats per setting, one day, and one pair of machines. A pair with a different speed ratio may behave differently, and other uneven workloads may have other task-size spreads. Because of the laptop's moving power limit and only three repeats, small differences should not be read as findings. Our solver was not compared with other parallel game-tree search methods. We did not separate the cost of unknown task sizes from the cost of changing speed in V0, and we did not compare the work of the laptop's fast and slow cores, because the solver does not record which core ran a task. Wi-Fi delay was measured with ping at the start of each round, not during every run. The sensor logger reports the average clock while the CPU runs rather than an effective clock, and the AMD machine has no power-limit or throttling data. The laptop's power limit is read from two interfaces and the lower value holds. The second machine changed during the project: we first planned a laptop (an HP EliteBook with an 8th-generation Intel Core i5, running Windows) and used the desktop PC instead when it became unavailable, with Linux on both machines. Two of our explanations, the PC's single memory channel and the cause of the laptop's changing power limit, are untested hypotheses.

VI. Conclusion and Future Directions

We built a distributed Connect Four solver and measured four ways of sharing its very uneven work between a hybrid-core laptop and a desktop PC that differ in processor, memory and cooling. Dealing out work as machines become free, together with splitting long tasks, cancelling unneeded ones and sharing solved positions, was clearly better than a fixed split decided from a speed measured at the start: V3 was about 1.8 times faster than V0 and about twice as fast as either machine alone. V0 and V1 kept every thread busy, but 66 to 70% of their positions went into stopped, cancelled or unneeded tasks, against 35 to 39% for V3. The laptop's speed changed during the runs because its power limit moved, and all unusually slow runs coincided with the low limit. The network made no clear difference. Splitting the search is costly: the machines examined many times more positions than a single unsplit search, and our best two-machine time was only about as fast as that search on one thread. In short, smart sharing clearly beats naive sharing, but the cost of splitting limits what a second machine can add for this problem.

Future work could schedule work from the live power limit and temperature of each machine, share search bounds between tasks so that less pruning is lost, compare our strategies with other parallel game-tree search methods, and use more machines, more start positions and more repeats. A study of the cores of the hybrid laptop separately, with threads tied to core types, would also address the multi-core side of the problem more directly.

References

[1] T. Čížek, M. Balko, and M. Schmid. "Massively parallel proof-number search for impartial games and beyond". In: *Proc. AAAI Conference on Artificial Intelligence (AAAI)*. arXiv:2511.10339. 2026. URL: https://arxiv.org/abs/2511.10339.

[2] T.-R. Wu et al. "Game solving with online fine-tuning". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36. arXiv:2311.07178. 2023. URL: https://arxiv.org/abs/2311.07178.

[3] J. B. Fernandes et al. *Adaptive asynchronous work-stealing for distributed load-balancing in heterogeneous systems*. 2024. arXiv: 2401.04494. URL: https://arxiv.org/abs/2401.04494.

[4] D. Beserra, E. D. Moreno, and J. Araujo. "Evaluating OpenMP scheduling and calibration strategies on asymmetric multicore processors: A Monte Carlo case study". In: *Advanced Information Networking and Applications (AINA 2026)*. Vol. 294. Lecture Notes on Data Engineering and Communications Technologies. Cham, Switzerland: Springer, 2026, pp. 357–369. DOI: 10.1007/978-3-032-23260-1_30.

[5] John Tromp. *John's Connect Four Playground*. URL: https://tromp.github.io/c4/c4.html (visited on Oct. 7, 2026).

[6] Pascal Pons. *Solving Connect 4: How to build a perfect AI*. Source code: https://github.com/PascalPons/connect4. 2016. URL: http://blog.gamesolver.org/ (visited on Oct. 7, 2026).